

Evaluación 1 – Programación 3

Objetivo

El objetivo de esta evaluación es consolidar los conocimientos adquiridos en las primeras unidades del cursado (HTML, CSS, JavaScript y TypeScript), mediante la evolución del proyecto Food Store hacia una aplicación frontend más dinámica e interactiva.

Consigna

A partir del proyecto desarrollado en los Trabajos Prácticos de:

- HTML
- CSS
- JavaScript
- TypeScript

deberás extender la aplicación actual incorporando las siguientes funcionalidades:

☒ Requerimientos obligatorios

1. Carrito básico con persistencia

Implementar un carrito de compras utilizando localStorage que permita:

- Agregar productos desde el catálogo
- Visualizar los productos agregados en una vista de carrito
- Mostrar nombre, precio y cantidad de cada producto
- Calcular y mostrar el total de la compra

! No es necesario implementar checkout ni conexión con backend.

Requerimiento técnico: Configuración del entorno

Repo base

El proyecto debe desarrollarse sobre el repositorio base provisto por la cátedra, que ya incluye Vite + TypeScript configurados. Cloná el repositorio y trabajá sobre esa estructura:

```
# 1. Clonar el repositorio base
git clone https://github.com/chiro45/proteger_rutas
cd proteger_rutas

# 2. Instalar dependencias con pnpm
pnpm install

# 3. Levantar el servidor de desarrollo
pnpm dev
```

El servidor de desarrollo estará disponible en <http://localhost:5173>.

🔗 Se recomienda usar pnpm como gestor de paquetes (ya configurado en el repo).

Si no lo tenés instalado: `npm install -g pnpm`

Estructura esperada

Las páginas y lógica del parcial deben ubicarse dentro de `src/` respetando la organización del proyecto:

```
src/
├── pages/
│   ├── store/
│   │   ├── home/
│   │   │   ├── home.html      ← catálogo de productos
│   │   │   └── home.ts        ← lógica: render, búsqueda, filtros
│   │   └── cart/
│   │       ├── cart.html      ← vista del carrito
│   │       └── cart.ts        ← lógica: render, cantidades, total
│   └── types/
│       ├── product.ts         ← interfaces Product y CartItem
│       └── categoria.ts       ← interface Icategoria
└── data/
    └── data.ts                ← PRODUCTS y getCategories()
```

Registrar las páginas en vite.config.ts

Vite necesita conocer todos los archivos HTML del proyecto para incluirlos en el build. Debés agregar tus páginas en la sección `build.rollupOptions.input` de `vite.config.ts`:

```
import { defineConfig } from 'vite';
import { resolve } from 'path';

export default defineConfig({
  build: {
    rollupOptions: {
      input: {
        // páginas existentes del repo base...
        index: resolve(__dirname, 'index.html'),

        // ☒ agregó tus páginas del parcial:
        storeHome: resolve(__dirname, 'src/pages/store/home/home.html'),
        storeCart: resolve(__dirname, 'src/pages/store/cart/cart.html'),
      },
    },
  },
  base: './',
});
```

- ❗ Si no registrás una página en `vite.config.ts`, no será incluida en el build y no podrá navegarse correctamente al ejecutar `pnpm build`.

◆ 2. Búsqueda y filtrado de productos

Incorporar funcionalidades de interacción sobre el catálogo:

- Búsqueda de productos por nombre
- Filtrado por categoría (desde el menú lateral o equivalente)

- ❗ No se requiere implementar ordenamientos ni combinaciones avanzadas de filtros.
-

Consideraciones técnicas

- El desarrollo debe realizarse exclusivamente con:
 1. HTML5
 2. CSS3
 3. JavaScript
 4. TypeScript
 - No se permite el uso de frameworks (React, Angular, etc.)
 - Los datos pueden mantenerse en arrays locales o en localStorage
 - Se debe respetar la estructura del proyecto trabajada en los TP
-

Entrega

La entrega deberá incluir:

1. Repositorio

- El código debe estar en un único .zip con un .txt con el link del video
 - El proyecto debe ser funcional y ejecutable
 - Debe incluir un archivo README.md con:
 - Descripción breve del proyecto
 - Instrucciones para ejecutarlo
-

2. Video de presentación (obligatorio)

Se deberá entregar un video con las siguientes características:

-  Duración: entre 10 y 15 minutos
 -  Cámara encendida durante toda la exposición
 -  Audio claro y comprensible
-

Contenido del video

En el video deberás:

1. Presentarte brevemente
2. Mostrar el funcionamiento de la aplicación:

- Catálogo de productos
 - Carrito
 - Búsqueda y filtros
3. Explicar:
- Qué funcionalidades lograste implementar
 - Cómo abordaste la resolución
 - Qué dificultades encontraste (si las hubo) y cómo las resolviste
-

I Criterios de evaluación

Se tendrá en cuenta:

- Correcto funcionamiento de las funcionalidades solicitadas
 - Uso adecuado de JavaScript y TypeScript
 - Manipulación del DOM
 - Uso de localStorage
 - Organización del código
 - Claridad en la explicación del video
 - Presentación general del proyecto
-

Importante

- No se evaluará la conexión con backend en esta instancia
 - El foco está puesto en la lógica frontend y la interacción con el usuario
-

Historias de Usuario de Referencia para esta Evaluación

HU-P1-01: Buscar productos en el catálogo

Como cliente de Food Store,
quiero poder buscar productos por nombre dentro del catálogo,
para encontrar más rápido el producto que deseo visualizar.

Criterios de aceptación:

- Debe existir un campo de búsqueda visible en la pantalla principal del catálogo.
- Al escribir un texto, se deben mostrar únicamente los productos cuyo nombre coincida total o parcialmente con la búsqueda.
- Si no hay coincidencias, se debe informar visualmente al usuario.
- La búsqueda debe realizarse sobre los productos cargados dinámicamente.

Dónde implementarlo:

- Vista principal del catálogo (`index.html` o su equivalente dentro de `pages/store/home/`).
- Lógica en el archivo JS/TS que renderiza los productos.

HU-P1-02: Filtrar productos por categoría

Como cliente de Food Store,
quiero poder filtrar los productos por categoría,
para visualizar únicamente aquellos que pertenecen al tipo de comida que me interesa.

Criterios de aceptación:

- Las categorías deben mostrarse en el menú lateral o sección equivalente.
- Al seleccionar una categoría, deben mostrarse solo los productos de esa categoría.
- Debe poder volver a visualizarse el catálogo completo.
- El filtrado debe aplicarse sobre los datos ya cargados en memoria o en `localStorage`.

Dónde implementarlo:

- Listado de categorías del catálogo.
- Misma lógica de renderizado dinámica utilizada para cargar productos y categorías.

HU-P1-03: Agregar productos al carrito

Como cliente autenticado,
quiero poder agregar productos al carrito desde el catálogo,
para ir armando mi compra.

Criterios de aceptación:

- Cada producto debe tener una acción visible para agregarlo al carrito.
- Al agregar un producto, este debe guardarse en `localStorage`.
- Si el producto ya fue agregado previamente, debe actualizarse su cantidad en lugar de duplicarse como ítem separado.
- Debe existir algún indicador visual de que la acción se realizó correctamente.

Dónde implementarlo:

- Botón “Agregar” dentro de cada tarjeta de producto renderizada dinámicamente.
- Lógica de carrito en un módulo, utilitario o archivo JS/TS separado.

HU-P1-04: Visualizar el carrito

Como cliente autenticado,
quiero visualizar los productos que fui agregando al carrito,
para revisar mi selección antes de una futura compra.

Criterios de aceptación:

- Debe existir una vista o página de carrito accesible desde la navegación.
- En ella deben mostrarse, como mínimo:
 - nombre del producto,
 - precio,
 - cantidad.
- Si el carrito está vacío, debe mostrarse un mensaje indicándolo.
- La información debe recuperarse desde `localStorage`.

Dónde implementarlo:

- Página `carrito` o vista equivalente dentro del frontend.
- Navegación desde el catálogo hacia el carrito.

HU-P1-05: Calcular el total del carrito

Como cliente autenticado,
quiero ver el total acumulado de los productos cargados en el carrito,
para conocer el importe total de mi selección.

Criterios de aceptación:

- En la vista del carrito debe mostrarse el total general.
- El total debe calcularse como la suma de todos los subtotales de los productos agregados.
- El valor debe actualizarse correctamente según el contenido almacenado.

Dónde implementarlo:

- Vista del carrito.
- Función de cálculo dentro de la lógica del carrito.